Introduction to Digital Logic Design Lab
EECS 31L
Lab 5: Processor

Ali Mortada
19849621

March 19th, 2025

---

1 Objective

The purpose of this lab was to design the upper module of the RISC-V processor that encloses its three sub-modules: the datapath, controller, and ALU controller. The first sub-module was designed in Lab 4, while the latter two were designed in this lab. Fig. 1 shows the processor's block diagram.

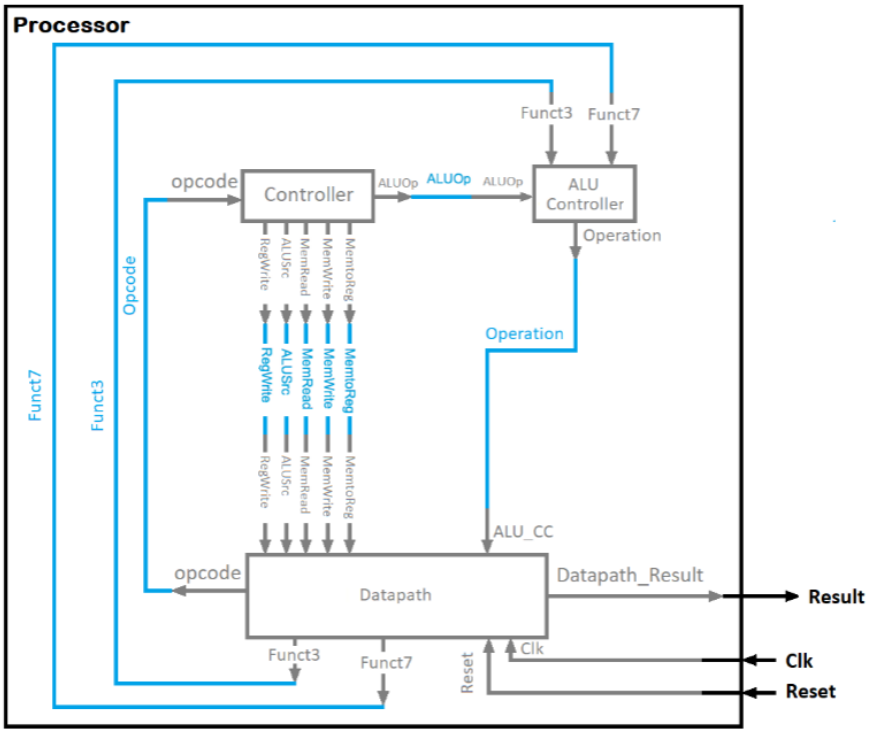

Figure 1: RISC-V processor block diagram.

Below, in Fig. 2, is the complete instruction set implemented in previous labs.

Funct7			Funct3		Opcode		
Imm[11:5]	Rs2	Rs1	010	Imm[4:0]	0100011	SW	S-type
Imm[11:0]		Rs1	010	rd	0000011	LW	I-type
Imm[11:0]		Rs1	000	rd	0010011	ADDI	
Imm[11:0]		Rs1	010	rd	0010011	SLTI	
Imm[11:0]		Rs1	100	rd	0010011	NORI	
Imm[11:0]		Rs1	110	rd	0010011	ORI	
Imm[11:0]		Rs1	111	rd	0010011	ANDI	
0000000	Rs2	Rs1	000	rd	0110011	ADD	R-type
0100000	Rs2	Rs1	000	rd	0110011	SUB	
0000000	Rs2	Rs1	010	rd	0110011	SLT	
0000000	Rs2	Rs1	100	rd	0110011	NOR	
0000000	Rs2	Rs1	110	rd	0110011	OR	
0000000	Rs2	Rs1	111	rd	0110011	AND	

Figure 2: Processor instruction set.

2 Procedure

2.1 Controller

The controller module takes as input the 7-bit `Opcode` field of instruction generated by the datapath module and has six outputs. The first output, `ALUOp`, is a 2-bit control signal that is passed into the ALU controller, while the other five outputs, `MemtoReg`, `MemWrite`, `MemRead`, `ALUSrc`, and `RegWrite` are 1-bit control signals for the datapath. Below, in Fig. 3, is the controller's block diagram.

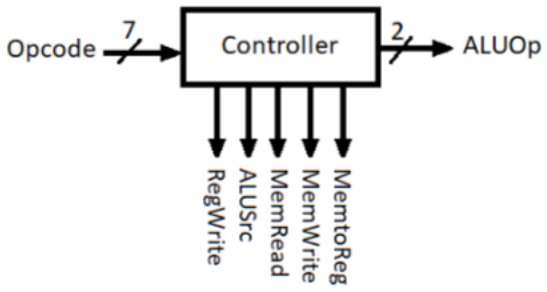

Figure 3: Controller block diagram.

There are four combinations of the `Opcode` input with each having a specific combination of values for the output control signals. This information is given in Fig. 4.

		Opcode			
		0110011	0010011	0000011	0100011
		AND, OR, ADD, SUB, SLT, NOR	ANDI, ORI, ADDI, SLTI, NORI	LW	SW
Control Signals	MemtoReg	0	0	1	0
	MemWrite	0	0	0	1
	MemRead	0	0	1	0
	ALUSrc	0	1	1	1
	Regwrite	1	1	1	0
	ALUOp	10	00	01	01

Figure 4: Control signals.

This behavior was defined using sequential logic – namely, using a `case` statement dependent on the `Opcode` input. Since each input has a specific set of outputs, the implementation is straightforward. Non-blocking statements were used to ensure that all of the assignments contained in each case happen concurrently. Below is the code used to define the module's behavior:

```
always @(Opcode) begin
    case (Opcode)
        7'b0110011:
            begin
                MemtoReg <=  1'b0;
                MemWrite <=  1'b0;
                MemRead  <=  1'b0;
                ALUSrc   <=  1'b0;
                RegWrite <=  1'b1;
                ALUOp    <= 2'b10;
            end
        7'b0010011:
            begin
                MemtoReg <=  1'b0;
                MemWrite <=  1'b0;
                MemRead  <=  1'b0;
                ALUSrc   <=  1'b1;
                RegWrite <=  1'b1;
                ALUOp    <= 2'b00;
            end
        7'b0000011:
            begin
                MemtoReg <=  1'b1;
                MemWrite <=  1'b0;
```

```
                MemRead  <=  1'b1;
                ALUSrc   <=  1'b1;
                RegWrite <=  1'b1;
                ALUOp    <= 2'b01;
            end
        7'b0100011:
            begin
                MemtoReg <=  1'b0;
                MemWrite <=  1'b1;
                MemRead  <=  1'b0;
                ALUSrc   <=  1'b1;
                RegWrite <=  1'b0;
                ALUOp    <= 2'b01;
            end
    endcase
end
```

2.2 ALU Controller

The ALU controller has two inputs `Funct3` and `Funct7` from the datapath and one input `ALUOp` from the controller. It has one 4-bit output `Operation` which is fed into the datapath and defines the ALU operation to use inside the datapath. Fig. 5 shows the ALU controller's block diagram.

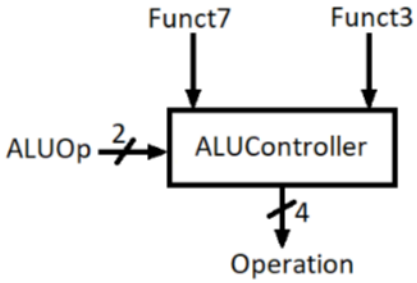

Figure 5: ALU controller block diagram.

Fig. 6 shows the instructions and their operation codes. However, a relation between the module's inputs and the output is needed, so the truth table is shown in Fig. 7.

operation	Operation code
AND, ANDI	0000
OR, ORI	0001
ADD, ADDI, SW, LW	0010
SUB	0110
SLT, SLTI	0111
NOR, NORI	1100

Figure 6: Instructions and their operation codes.

	Funct7	Funct3	ALUop	Operation			
AND	0000000	111	10	0	0	0	0
OR	0000000	110	10	0	0	0	1
NOR	0000000	100	10	1	1	0	0
SLT	0000000	010	10	0	1	1	1
ADD	0000000	000	10	0	0	1	0
SUB	0100000	000	10	0	1	1	0
ANDI	-	111	00	0	0	0	0
ORI	-	110	00	0	0	0	1
NORI	-	100	00	1	1	0	0
SLTI	-	010	00	0	1	1	1
ADDI	-	000	00	0	0	1	0
LW	-	010	01	0	0	1	0
SW	-	010	01	0	0	1	0

Figure 7: Truth table for the 4-bit operation code using the ALU controller module's inputs.

Again, since the truth table is given, the module's behavior can be implemented using a `case` statement dependent on the `Funct7` input. Inside each case, nested ternary statements can be used to determine the output based on the other two inputs, `Funct3` and `ALUOp`. The main difference is that a `casex` statement is needed to take care of the cases where `Funct7` can be any value, which are denoted with a "-" in Fig. 7. Below is the code used to implement the module's behavior.

```
always @(*) begin
    casex (Funct7)
        7'b0000000:
        begin
            Operation <= ((Funct3 == 3'b111) && (ALUOp == 2'b10)) ? 4'b0000 :
                         ((Funct3 == 3'b110) && (ALUOp == 2'b10)) ? 4'b0001 :
                         ((Funct3 == 3'b100) && (ALUOp == 2'b10)) ? 4'b1100 :
                         ((Funct3 == 3'b010) && (ALUOp == 2'b10)) ? 4'b0111 :
                         4'b0010;
        end
        7'b0100000:
        begin
            if ((Funct3 == 3'b000) && (ALUOp == 2'b10))
                Operation <= 4'b0110;
        end
        7'bxxxxxxx:
        begin
            Operation <= ((Funct3 == 3'b111) && (ALUOp == 2'b00)) ? 4'b0000 :
                         ((Funct3 == 3'b110) && (ALUOp == 2'b00)) ? 4'b0001 :
                         ((Funct3 == 3'b100) && (ALUOp == 2'b00)) ? 4'b1100 :
                         ((Funct3 == 3'b010) && (ALUOp == 2'b00)) ? 4'b0111 :
                         ((Funct3 == 3'b000) && (ALUOp == 2'b00)) ? 4'b0010 :
                         ((Funct3 == 3'b010) && (ALUOp == 2'b01)) ? 4'b0010 :
                         4'b0010;
```

```
        end
    endcase
end
```

2.3 Processor

Now that all of the sub-modules were designed, the top module connecting them was created. The module was designed using the block diagram in Fig. 1. The blue lines were defined as internal wires within the module and used to connect the sub-modules. Below is the code used to define the module's behavior.

```
// internal wire declarations
wire MemtoReg;
wire MemWrite;
wire MemRead;
wire ALUSrc;
wire RegWrite;
wire [6:0] Opcode;
wire [1:0] ALUOp;
wire [2:0] Funct3;
wire [6:0] Funct7;
wire [3:0] Operation;

// instantiate and connect each module
Controller ctrl(
    .Opcode(Opcode),
    .ALUSrc(ALUSrc),
    .MemtoReg(MemtoReg),
    .RegWrite(RegWrite),
    .MemRead(MemRead),
    .MemWrite(MemWrite),
    .ALUOp(ALUOp)
);

ALUController alu_ctrl(
    .ALUOp(ALUOp),
    .Funct3(Funct3),
    .Funct7(Funct7),
    .Operation(Operation)
);

data_path dp(
    .clk(clk),
    .reset(reset),
    .reg_write(RegWrite),
    .mem2reg(MemtoReg),
    .alu_src(ALUSrc),
    .mem_write(MemWrite),
    .mem_read(MemRead),
    .alu_cc(Operation),
    .opcode(Opcode),
    .funct3(Funct3),
```

```
        .funct7(Funct7),
        .alu_result(Result)
    );
```

3 Simulation Results

3.1 Processor

The testbench code provided in the lab manual was used to test the processor module. The code defines 20 test cases and a `point` variable which increments each time a test case is passed. Below, in Fig. 8, is a screenshot of the simulation result:

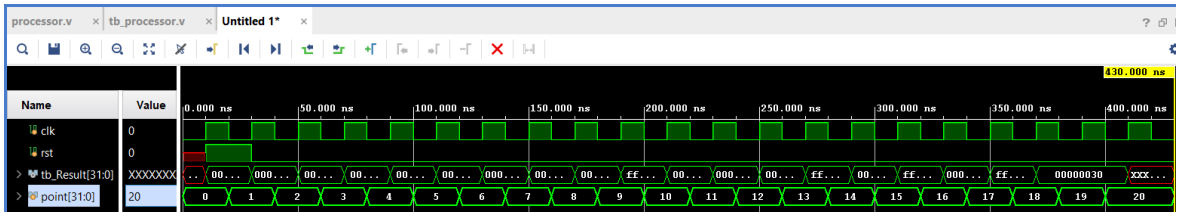

Figure 8: Processor testbench simulation result.

As shown in the screenshot, the final value of the `point` value is 20 (converted from hexadecimal to decimal for easier readability), meaning the module passed each test case.